SED
a division of CALIAN Ltd.

68192637

Revision 18
August 4, 2016

Sirius XM Radio
Metadata
Interface Control Document

128149

Prepared by	Systems Specialist	B. Willenborg
Prepared by	Systems Analyst	R. Hancock
Approved by	Systems Team Lead	M. Gale
Approved by	Software Team Lead	T. Huenison
Approved by	Project Manager	A. Sims

18 Innovation Boulevard
Saskatoon, Saskatchewan CANADA S7N 3R1
Tel: (306) 931-3425 Fax: (306) 933-1486
www.sedsystems.ca

Contents

Contents (Cont'd)

Contents (Cont'd)

Page

Figures

Tables

1.0 Introduction

1.1 Identification

This document is identified as:

XM Radio Metadata Interface Control Document
SED Document No. 128149

1.2 Purpose

This document defines the XM Program Associated Data (PAD) interface and the Sirius Program Descriptive Text (PDT) interface, as implemented by the Uplink Delivery System (UDS) and Mcommerce PAD/PDT Delivery System (MPDS). Henceforth, the term "metadata" is used in this document to refer to PAD and PDT in a general sense.

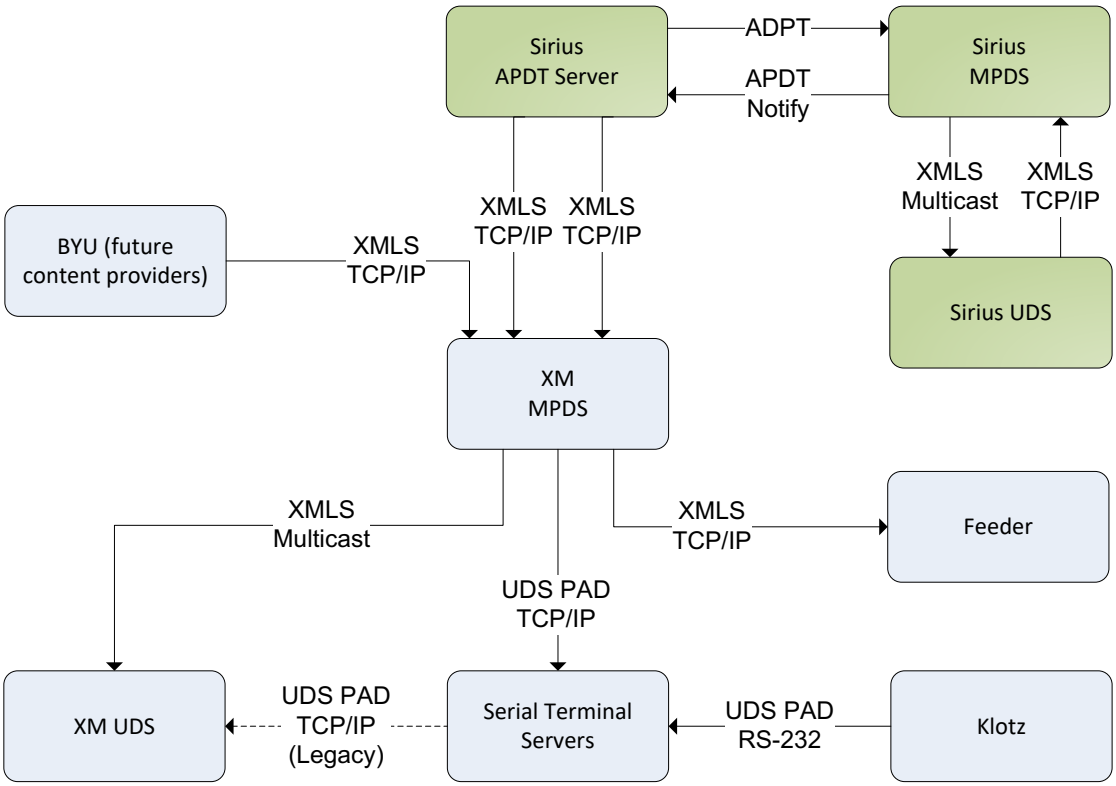

Figure 1-1 MPDS External Interfaces

1.3 Applicable Documents

ID	Document	Title	Source
AD0	XM-Sirius Content Interface ICD V1.3	XM-Sirius Content Interface ICD	SXM
AD1	SX-9845-0192	PID (Program ID) Coding for XM Band	SXM
AD2		Satellite Digital Audio Radio Service (SDARS) Asynchronous PDT	SXM

1.4 Change History

Revision	Change Description
1	Initial issue
2	Added artist_id field to DM message
3	Added information from XM Satellite Radio MPDS EICD (SED 128002). Added next-song lookahead support (DN automation PAD message, new queue_order XML-S attribute). Added automatic PID generation algorithm to document. Added composer field that originates out of Dalet into the DM/DN message.
4	Clarified effect of auto program ID generation on B-4 message. Added details of UDS extended artist/song handling. Added note about UDS usage of DM/DN messages. Updated descriptions of event ID and program ID mapping to match current MPDS implementation. Added note about handling of duplicate fields in DM/DN message. Reworded XML-S description to not be restricted to forwarding only to a PDT server. Truncate composer field when translating from DM to XMLS.
5	Added SID field to DM/DN and XML-S messages.
6	Removed A1, A2, A3, AR, B3, B4 and D1 messages. Added note regarding handling of zero program ID in B-4, DM and DN messages. Updated max length of all PAD labels to 36 characters to match Service Layer Specification release 3.2. Removed XM PID format details as they are now contained within AD1. Added layer, duration and start_offset fields to DM/DN and XML-S. Removed DirecTV/XML-D format details as it is no longer used. Removed SID field from DM/DN as is used only on the XML-S interface. When using DM as primary message source, basic labels are auto generated by truncating length.
7	Added Artist ID field encoding details. Added XML-S multicast. Deprecated MPDS support for automation PAD output. Removed HTTP PAD interface.
8	Added Sirius APDT interface details. Added note regarding XML-S multicast message repeats. Updated terminology to standardize on metadata instead of PAD or PDT. Reworked mappings to describe inputs and outputs separately for each interface type.
9	Resolved TBCs in APDT interface details. Added note that –L and –I options are to be specified by MPDS on outgoing APDT notification messages.
10	Added APDT assoc message.
11	APDT message length is exclusive of header section, not inclusive.

Revision	Change Description
12	Added XML-S field <record_enabled> to support DRM.
13	Add new XML-S fields for As-Played report.
14	Updated XML-S field names for Reconciliation ID, Content Type, and Asset ID.
15	LI#45110 – clarify ignored fields in B-4 message.
16	Added data format for Metadata Reference (MRef) and Look-ahead Metadata Reference (LAMRef) messages and updated PADMetadata message format.
17	Added mappings for MRef/LAMRef for SXM17. Updated outdated information to match current implementation.
18	Removed RFU from required fields for LAMRef and MRef messages.

1.5 Abbreviations

APDT	Asynchronous PDT
ASCII	American Standard Code for Information Interchange
BIC	Broadcast Information Channel
DRM	Digital Rights Management
ISO	International Organization for Standardization
LAMRef	Look-ahead Metadata Reference
MPDS	Mcommerce PAD Delivery System
MRef	Metadata Reference
PAD	Program Associated Data
PID	Program Identifier
PDT	Program Descriptive Text
UDS	Uplink Delivery System
XML	eXtensible Markup Language

This page intentionally left blank.

2.0 Interface Definition

2.1 Legacy XM Metadata Interface

Metadata feeds are received by Moxa serial device servers in RS-232 format, and the data is retrieved by the MPDS connecting to the Moxa devices using TCP/IP. The MPDS is also capable of generating messages in this format based on incoming messages (i.e. from an XML-S source). This capability is used by legacy (<=6.0) UDS systems in order to receive metadata from the MPDS. However, this capability is deprecated as of UDS 6.1 since the XML-S multicast interface will now be used for this purpose.

2.1.1 Physical Interface

- Electrical: RS-232, asynchronous.
- Transport Protocol: ASCII text.
- Data Rate: 4800 baud, 8N1, no flow control. (Note that these settings can be modified as part of the Moxa configuration, as long as the metadata source systems are configured accordingly; other systems do not depend on these settings.)

2.1.2 Logical Interface

2.1.2.1 Messages Transmitted by MPDS

Flow control (XON/XOFF) characters may be transmitted by the Moxa device if software flow control is enabled. Otherwise no feedback is provided.

2.1.2.2 Messages Received by MPDS

Metadata commands are encoded in text format using the Latin-1 (ISO-8859-1) character set. The command line terminator is the line feed character (ASCII #10) and the field separator is <STX> (ASCII #02). The old field separator which is a ':' (ASCII #58) is still supported, however use of this separator is discouraged because the colon character can be part of a valid message label, and it will cause message parsing problems if it is present there. Leading and trailing white space in a command line field is ignored (other than <STX>). (White space is defined as any non-printable ASCII character that is not recognized by the command interpreter.)

The first field in a command line is the Message Type which identifies the format of the remainder of the message.

The original format for metadata delivery was to use the B-4 (Artist/Song), A4 (Extended Song Label) and A5 (Extended Artist Label) messages. The DM and DN messages were added later to provide additional program information. As part of the extended metadata updates to the UDS it is now possible to configure each individual channel to use solely the DM and DN messages and ignore B-4, A4 and A5 messages. The UDS can still be configured to use the older messages, in the event that a particular channel does not yet supply valid DM/DN information.

The older message types should not be used for new system implementations. Going forward, the DM and DN messages are preferred since they are extensible in a backward-compatible fashion, are more robust due to checksum/message length verification, and provide all necessary information regarding a program in an atomic fashion.

For all message types, repeats of the same message are not necessary and will have no effect.

If the message type is valid, then the remainder of the command line is interpreted in accordance with the message type as described in the following table. Otherwise the entire message is discarded (up to the next line feed character). In all cases, an invalid entry in any field results in the entire command line being abandoned.

Message Type	Field	Data Type	Description
Extended Song Label Message (New implementations should use DM format) Note: This message should be sent after the corresponding B-4 message providing the basic song label for the current song. Otherwise it will be considered to be associated with the previous B-4 message.			
A4	New Message Flag	Numeric	Set to 1 for a new message or 0 for a repeat message.
	Song Label	Text	Up to 36 ASCII characters.
Extended Artist Label Message (New implementations should use DM format) Note: This message should be sent after the corresponding B-4 message providing the basic artist label for the current song. Otherwise it will be considered to be associated with the previous B-4 message.			
A5	New Message Flag	Numeric	Set to 1 for a new message or 0 for a repeat message.
	Artist Label	Text	Up to 36 ASCII characters.
Artist/Song Label Message (New implementations should use DM format)			
B-4	Advanced Application Control	Numeric	0 = Recording Enabled 1 = Recording Disabled
	Duration Progress Format	Numeric	Ignored
	Duration	Numeric	Ignored
	Progress	Numeric	Ignored
	Display #1 Size	Numeric	Ignored
	Display #2 Size	Numeric	Ignored
	Artist Mask 1	Text	Ignored
	Artist Mask 2	Text	Ignored
	Artist Label	Text	Up to 16 characters.
	Song Mask 1	Text	Ignored
	Song Mask 2	Text	Ignored
	Song Label	Text	Up to 16 characters.
	Program ID	Numeric	Range 0 to 4294967296. Note that if auto program ID generation is enabled for the service component on the UDS, it will ignore this value and use an internally generated value instead. If a zero program ID is provided, the UDS will retain the last non-zero program ID provided in a previous artist/song label message.

Message Type	Field	Data Type	Description
Metadata Message **Note: The DM and DN messages have the same format. DM refers to the current song being played while DN refers to the next song to be played.** **DM messages should be sent before the corresponding DN message, as DN message data may be invalidated when a new DM message is received.** **This message is actively used by the UDS and MPDS for transmitting output metadata (unless the channel is configured to use the B-4/A4/A5 artist/song messages instead).**			
DM (current song) DN (next song)	Length	Numeric	Length of the Data string (no more than 4 characters)
	Checksum	Text	Hex of the checksum (computed on the Data string only), 2 chars. The checksum is computed as an exclusive OR on all the bytes of the data string.
	Data string	Text	Up to 2000 characters. (Typically 300 bytes long) The data is formatted as a set of parameter-value pairs using URL encoding (as with an HTTP POST request), for example: event_id=1234567&title_id=987654321 As with URL encoding, any reserved characters should be escaped using percent encoding as per RFC 3986, for example the & character is escaped as %26. Note that only the field data values are URL encoded. See Table 2-1 for details. The absence of a mandatory field invalidates the message. Default values are assumed for any non-mandatory fields that are missing. Undefined fields are ignored. If a field occurs more than once in the same message, the value provided in the last occurrence will be used.
Heartbeat Message			
H0			Used to maintain the connection if other metadata messages are not being sent. The UDS/MPDS will raise an alarm if no metadata message is received within a configured time period.

Table 2-1 DM/DN Message Field Details

Name	Parameter Name	Mandatory	Broadcast/Automation	Default Value
Event ID	event_id	Yes	The first 9 digits, of the schedule event reference number (event_num2) field, should be used to populate that field.	N/A
Title ID	title_id	No	No data needed	N/A
Item Code	item_code	No	Item_ID should be used for that field.	N/A
Artist	artist	Yes	This should be taken from textout1 (artist) for the first 16 characters and a new textout field should be added (say textout3) for the remaining 16 characters. Trailing space should be removed. Maximum length 36 characters.	N/A
Song Title	song_title	Yes	This should be taken from textout2 (song) for the first 16 characters and a new textout field should be added (say textout4) for the remaining 16 characters. Trailing space should be removed. Maximum length 36 characters.	N/A
Album	album	Yes	Album name for the given song/program. Maximum length 36 characters.	N/A
Channel	channel	No	BUS ID should be used. This field will become mandatory in an upcoming release.	<blank>
SKU	sku	No	No data needed	<blank>
ISRC	isrc	No	International Standard Recording Code (ISRC) for the given song/program.	<blank>
Program ID	program_id	Yes	Program ID (decimal) for the given song as defined in AD1. Zero means that last received PID is to be preserved.	N/A
Music DB ID	music_db_id	No	List of IDs for music databases. Field separator is the pipe "|" character. Dalet encodes both the AMG ID and the iTunes Song ID into this field using the pipe character '|' as the sub-field separator. The parsing rules for Music DB ID are: If there is no AMG track ID and no iTunes Song ID, the field may or may not have the "|" (PIPE) character. If there is no AMG track ID and there is an iTunes ID, you will have the "|" (PIPE) followed by the iTunes ID. If there is an AMG track ID and no iTunes ID, you will have the AMG track ID either followed by the "|" (PIPE) or not followed by the "|" (PIPE).	<blank>
Label	label	No	Record label for the song/program. Maximum length 36 characters.	<blank>
Composer	composer	No	This should be taken from the composer field stored in Dalet. Maximum length 36 characters.	<blank>
Sirius PID	sirius_pid	No	The lower-band program ID for the song/program.	<blank>
Artist ID	artist_id	No	Low Band Artist ID field for the song/program. 32-bit integer encoded as described in Section 2.1.2.4.	<blank>
Record Enabled	record_enabled	No	1 if recording is enabled for this program, 0 if not.	1
Duration	duration	No	Duration of the indicated program in seconds.	0
Start Offset	start_offset	Yes (for DN)	Time (in seconds) until the indicated program begins. Applicable to DN message only. Ignored for DM as the given program is assumed to begin immediately.	N/A

2.1.2.3 Extended Artist/Song Label Handling

When using the legacy B-4/A4/A5 messaging for artist/song labels, the UDS and MPDS must infer an association between the labels provided by the artist/song label message (B4 or B-4) and the extended artist and song label (A4/A5) messages. The following rules are applied to do this:

- Extended label messages are deemed to be associated with the last received basic label message. This means that the basic label message for a given song should always be sent before the extended label(s).
- When a new basic label message is received, any previously received extended artist/song label messages will be invalidated (as they are considered to have been associated with the previous song).
- If a new extended label message is received without having received a new basic label, it will be considered an update to the extended label for the current song.
- If either the artist or song in the basic label message is blank, but the corresponding extended message received is not blank, the UDS/MPDS will place the first 16 characters of the extended label in the basic label. Note that this feature has some side effects, such as multiple BIC updates on the UDS, and is considered a legacy workaround that new or updated systems should not depend upon.
- The BIC messages sent by the UDS are generally more space-efficient if the basic label is equal to the first 16 characters of the extended label, since the BIC extended label message can be sent in append mode rather than replace mode. Therefore, spurious differences between the text in the basic and extended labels should be avoided.

When the DM/DN messages are used for metadata, the UDS will automatically set the basic artist/song labels based on the first 16 characters of the provided labels. The extended label BIC messages will not be transmitted if the label is 16 characters or less. The artist and song label masks will be set with mask 1 being 8 characters and using the first 8 characters of the label and mask 2 being 10 characters and using the first 10 characters of the label.

2.1.2.4 Artist ID Encoding

The artist ID field in the DM/DN messages is encoded in a special format. It is similar to Base64 encoding but uses a different character mapping.

The 32-bit binary value is encoded by converting chunks of up to 6 bits into the corresponding ASCII character as shown in the table following.

Character	Value	Character	Value	Character	Value	Character	Value
#	62	F	15	V	31	k	46
0	0	G	16	W	32	l	47
1	1	H	17	X	33	m	48
2	2	I	18	Y	34	n	49
3	3	J	19	Z	35	o	50
4	4	K	20	_	63	p	51
5	5	L	21	a	36	q	52
6	6	M	22	b	37	r	53
7	7	N	23	c	38	s	54
8	8	O	24	D	39	t	55
9	9	P	25	e	40	u	56
A	10	Q	26	f	41	v	57
B	11	R	27	g	42	w	58
C	12	S	28	h	43	x	59
D	13	T	29	i	44	y	60
E	14	U	30	j	45	z	61

The 6-bit chunks of the 32-bit value are converted into characters following the mapping below:

Destination Character Number	Source Bits
1	30-31 (top 4 bits treated as 0)
2	24-29
3	18-23
4	12-17
5	6-11
6	0-5

Leading 0 characters are then removed, resulting in a string between 1 and 6 characters in length.

Some examples of the encoding are shown below.

Encoded String	Decimal Value
7CM	29462
7QK	30356
3_____	4294967295

2.1.2.5 Examples

```
A4<STX>1<STX>Dream On
```

This indicates a new extended song label set to “Dream On”.

```
A5<STX>0<STX>Aerosmith
```

This indicates a new extended artist label set to “Aerosmith”.

```
B-4<STX>1<STX>0<STX>532<STX>120<STX>5<STX>12<STX>100011011
<STX>111111111<STX>Aerosmith<STX>11000011<STX>11111111<STX>Dream
On<STX>123456
```

The advanced application control field indicates that recording is disabled. There are two displays: one with 5 characters and one with 12 characters. The artist appears as “Asmth” on the first display and “Aerosmith” on the second display. The song title appears as “Dr On” on the first display and “Dream On” on the second display. The program ID value is 123456.

```
DM:163:29:album=I%20Used%20to%20Wander%20These%20Streets&artist=Billie%20The%
20Vision%20And%20The%20Dancer&event_id=100106004A8F40&program_id=4886336&song
_title=Relay%20Race
```

This provides an event ID of 100106004A8F40, event ID of 100106004A8F40, artist of “Billie The Vision And The Dancer”, song title of “Relay Race”, and album of “I Used to Wander These Streets”.

```
DN:163:29:album=I%20Used%20to%20Wander%20These%20Streets&artist=Billie%20The%
20Vision%20And%20The%20Dancer&event_id=100106004A8F40&program_id=4886336&song
_title=Relay%20Race
```

This example is the same as the previous one, except that it indicates that the song details are for the next song rather than the current one.

```
H0
```

This message acts as a heartbeat to indicate that the connection is still active even though no meaningful messages are being sent.

2.2 XML-S TCP Metadata Interface

2.2.1 Connection Specification

The MPDS supports acting as a TCP/IP client or a server for the XML-S TCP interface. The same message format is used for either the client or the server to transmit messages. Metadata is only sent by one party per connection. The MPDS supports sending or receiving metadata for TCP/IP client connections and receiving metadata for TCP/IP server connections. The type of TCP/IP connection and the receiver/sender relationship is configurable in the MPDS.

2.2.2 MPDS Acting as a TCP/IP Server

When acting as the server, both the online and standby MPDS servers accept connection requests defined TCP port numbers. Clients are expected to connect to both the online and standby MPDS servers. Each client is limited to one connection per MPDS server per TCP port. If a new connection request is received from a connected client, the MPDS server will actively disconnect the existing connection. Each MPDS server handles the communications with the XML-S sources independently. Loss in communications with one MPDS server will not automatically impact the communications with the second MPDS server.

MPDS XML-S server ports can be defined as either containing the current metadata or containing "look ahead" content. A second TCP port is used to ensure that the real time messages are not delayed for future content. The second TCP port is functionally identical to the primary port, only that messages sent on the second port will have a queue value greater than 0 to indicate future content.

2.2.3 MPDS Acting as a TCP/IP Client

The MPDS also supports acting as a XML-S TCP/IP client. Only the online MPDS will attempt a connection to the server. In this mode it is configurable if the TCP/IP client or server will be sending metadata updates.

In addition to sending and receiving metadata for the current song, this interface also supports advance notice of metadata for future songs. The message contains an XML attribute to identify the content to which the information applies (i.e. current or future based on the value of the queue attribute).

2.2.4 Data Message Formats

Each message sent on the XML-S interface consists of separate XML documents of the form PAD Metadata, Metadata Reference (MRef), Look-ahead Metadata Reference (LAMRef), or HeartBeat. Each of these forms is defined below. Note that the order of the elements within these messages is not significant. The elements listed as required in these formats must be present or else the message will be rejected. Optional elements in these formats may be omitted – the value will be considered blank.

For backward compatibility, the recipient of an XML-S message should assume blank values for any field that is missing. **For forward compatibility, the recipient of an XML-S message should silently ignore undefined fields.**

Messages are expected to arrive at the time at which they are to be forwarded to the configured destinations. If there is a fixed latency between message arrival and when they should be forwarded, the MPDS supports delaying the forwarding by a fixed amount for all messages received on the XML-S TCP input. Note that there is no XML header at the start of the message. All text within the message should be represented in the ISO-8859-1 (Latin-1) character set (not UTF-8).

Each of these received XML messages will be followed by an ASCII STX (character code 2) character, which serves as a delimiter between messages.

2.2.4.1 Data Message Value Place-holders

The following sections describe the data message formats for PADMetaData, MRef, LAMRef, and HeartBeat as a set of fields and place-holders representing values. This section provides a list of those place-holders and describes the values they represent.

- **${Album}** is the album title.
- **${Artist}** is the artist name.
- **${Artist ID}** is the Sirius artist ID field for the message.
- **${Asset ID}** information for Business Intelligence As-Played reporting.
- **$(BIC MRef Hour)** is the position in a 4 hour rolling window. First element of the row address for the reported MRef in its associated data table.
- **${BIC MRef RFU}** is reserved for future use. This field shall be ignored by the receiver. This field shall be set to 0 until further notice.
- **${BIC MRef Transition ID}** is the current hour transition identifier (1 – 511) or clear (0). Second element of the row address for the reported MRef in its associated data table.
- **${BIC MRef Type}** is the Type or Class of the Metadata Reference.
  - 0 = Cut
  - 1 = Segment
  - 2 = Program
  - 3 = Reserved for future use
- **${Bus Code}** is the assigned XM/Sirius bus code corresponding to this message (normally 6 characters for full-time channels). It can be 1 to 8 alphanumeric characters.
- **${Composer}** is the composer for the song.
- **${Content Type}** information for Business Intelligence As-Played reporting.
- **${Duration}** is the program running time (in seconds. If not available, it is encoded as blank.
- **${Layer}** indicates which service layer contains this SID. 0 indicates the base layer (or legacy service) and 1 indicates the Overlay service.
- **${Music DB ID}** is the track ID from a 3rd party track identification system such as an AMG ID or an iTunes Song ID. See the definition of the music_db_id field in Table 2-1.
- **${Program ID}** is the Dalet Program ID (XM platform) or, in the case of a static metadata source, a value generated automatically by the MPDS in accordance with AD1 based on a per-source configurable player ID. This field will be blank if a program ID is not available for the content.

- **${Queue Order}** is the order in which the song will be played. A value of 0 indicates the current song. This is the default value if this tag is not present in the message. Non-zero values indicate a future song. The song on-deck will have a queue value of 1. A value of 2 indicates the song that will be played after that, and so on. Note that messages applicable at one moment in time should generally be sent in ascending queue order, as next-song information may be overwritten when a new current song is received.
- **${Reconciliation ID}** information for Business Intelligence As-Played reporting.
- **${Record Enabled}** indicates if recording of the program for playback at a later time is permissible. Choices are:
  - 0 = No
  - 1 = Yes, which is implied if this field is omitted**.**
- **${Service ID}** is the ID of the target service (channel). It can be numbers 1 to 255.
- **${SID}** is the upper-band service ID for the channel. Note that the MPDS treats this field as an output and not an input – it uses the value set in its configuration for the channel when outputting messages and does not propagate the received value.
- **${Sirius PID}** is the Sirius program ID field for the message. This is used to support advanced Sirius receiver features, such as the Jump Button, Virtual Categories, and Game Alert.
- **${Song Title}** is the song title.
- **${Start Offset}** indicates the time (in seconds) until the indicated program is scheduled to begin. For messages with queue_order of 0, the program is assumed to begin immediately.
- **${UTC Time Stamp}** is the RFC 3339 timestamp for the UTC time zone for when the program starts. If value is not given, now is assumed.

There is no inherent limit to the maximum length of the artist and song title labels other than a maximum total message size of 64KB. However, when received on the XM side by the MPDS and sent for broadcast over the air, the labels will be truncated to the first 36 characters. (Older radios will display only the first 16 characters.) Labels sent to XMRO from XML-S inputs will not be truncated by the MPDS.

Any XML reserved characters within the channel code, bus code, artist, or song title must be escaped according to the XML specification so that the XML remains parsable. The escapes applied by the MPDS for transmission are:

Character	Escape Sequence
'	&apos
“	&quot
&	&amp
<	&lt
>	&gt

2.2.4.2 PAD Metadata

```
<PADMetaData queue="${Queue Order}">
    <ts>${UTC Time Stamp}</ts>
    <start_offset>${Start Offset}</start_offset>
    <artist>${Artist}</artist>
    <songtitle>${Song Title}</songtitle>
    <channelcode>${Bus Code}</channelcode>
    <siriuspid>${Sirius PID}</siriuspid>
    <artistid>${Artist ID}</artistid>
    <album>${Album}</album>
    <composer>${Composer}</composer>
    <duration>${Duration}</duration>
    <programid>${Program ID}</programid>
    <musicdbid>${Music DB ID}</musicdbid>
    <sid>${SID}</sid>
    <layer>${Layer}</layer>
    <record_enabled>${Record Enabled}</record_enabled>
    <content_type>${Content Type}</content_type>
    <asset_id>${Asset ID}</asset_id>
    <rec_id>${Reconciliation ID}</rec_id>
</PADMetaData>
```

Table 2-2 PADMetadata Required Values

Field	Required
${Queue Order}	Yes
${UTC Time Stamp}	Only for messages with a queue order not equal to zero that do not define ${Start Offset}. Defining ${UTC Time Stamp} as opposed to ${Start Offset} is preferred.
${Start Offset}	Only for messages with queue_order other than 0 that do not define UTC Time Stamp. Defining ${UTC Time Stamp} as opposed to ${Start Offset} is preferred.
${Artist}	Yes
${Song Title}	Yes
${Bus Code}	Yes
${Sirius PID}	No
${Artist ID}	No
${Album}	No
${Composer}	No
${Duration}	No
${Program ID}	No
${Music DB ID}	No
${SID}	No
${Layer}	No
${Record Enabled}	No
${Content Type}	No
${Asset ID}	No
${Reconciliation ID}	No

Refer to 2.2.4.1 Data Message Value for a description of each value.

2.2.4.3 Metadata Reference (MRef)

```
<MRef bc="$(Bus Code)"
   sid="$(Service ID)"
   lay="$(Layer)"
   ts="$(UTC Time Stamp)"
   typ="$(BIC MRef Type)"
   hr="$(BIC MRef Hour)"
   tid="$(BIC MRef Transition ID)"
   rfu="$(BIC MRef RFU)" />
```

Table 2-3 MRef Required Values

Field	Required
${Bus Code}	Yes
${Service ID}	No
${Layer}	No
${UTC Time Stamp}	No
${BIC MRef Type}	Yes
${BIC MRef Hour}	Yes
${BIC MRef Transition ID}	Yes
${BIC MRef RFU}	No

Refer to 2.2.4.1 Data Message Value for a description of each value.

2.2.4.4 Look-ahead Metadata Reference (LAMRef)

```
<LAMRef bc="$(Bus Code)"
   sid="$(Service ID)"
   lay="$(Layer)"
   ts="$(UTC Time Stamp)"
   typ="$(BIC MRef Type)"
   hr="$(BIC MRef Hour)"
   tid="$(BIC MRef Transition ID)"
   rfu="$(BIC MRef RFU)" />
```

Table 2-4 LAMRef Required Values

Field	Required
${Bus Code}	Yes
${Service ID}	No
${Layer}	No
${UTC Time Stamp}	Yes
${BIC MRef Type}	Yes
${BIC MRef Hour}	Yes
${BIC MRef Transition ID}	Yes
${BIC MRef RFU}	No

Refer to 2.2.4.1 Data Message Value for a description of each value.

2.2.4.5 Heartbeat

The HeartBeat message content can be sent periodically if no other metadata is being sent. It has no purpose other than to maintain the connection. This allows the XML-S client to distinguish a failed connection from a legitimately idle one.

```
<HeartBeat>
    <channelcode>${Bus Code}</channelcode>
</HeartBeat>
```

Table 2-5 HeartBeat Required Values

Field	Required
${Bus Code}	No

Refer to 2.2.4.1 Data Message Value for a description of each value.

2.2.5 Error Handling

The connection will remain open even if the XML-S client is unable to parse a message. There is no error feedback to the sender. If the MPDS receives an invalid message (i.e. XML cannot be parsed, required elements missing, etc.) it will raise an alarm to indicate it has received an invalid message.

2.3 XML-S Multicast Metadata Interface

2.3.1 Connection Specification

The XML-S multicast metadata interface is used to forward metadata information between the MPDS and UDS. It may also be used by other systems in the future.

The message formatting is similar to the XML-S TCP metadata interface, but instead of sending individual XML documents separated by STX characters over a TCP connection, the individual XML documents will be sent as the payload of a multicast UDP packet. No additional header information is included. As with XML-S TCP, the PADMetaData, MRef, LAMRef, and HeartBeat elements are supported.

In order to ensure that the resulting packet fits within the standard Ethernet MTU of 1500 bytes without fragmentation and reassembly, each XML-S payload should be no larger than 1472 bytes (this accounts for the size of the IPv4 and UDP headers).

Note that since UDP message delivery is potentially unreliable, these messages may be lost in some cases. In order to mitigate this possibility, it is recommended that senders of these messages repeat the message on a periodic basis as long as the same metadata is in effect, so that the metadata will eventually update even if a packet is lost. This also allows a newly connected XMLS client to acquire the current state of affairs on startup (as there is no mechanism for the XML-S server to know when that happens.)

The multicast destination address and port are configurable on the MPDS. The selection of the address is outside of the scope of this document, but it should be ensured that it does not conflict with other multicast services used on the network. Use of the Administratively Scoped IPv4 Multicast address range as defined by RFC 2365, 239.0.0.0/8, is recommended.

Only the online MPDS will send messages. All channel data being output by the MPDS (including different queue order values) will normally be sent to the same destination address and port. As with the XML-S TCP interface, the XML-S client is expected to filter the received messages based on the queue order and the content of the ${Bus Code}element.

In order to ensure that it receives the traffic, in addition to opening a UDP listen socket on the appropriate port, the receiving software must take additional steps to join the IP multicast group for the destination address being used. (For example, under Linux, the IP_ADD_MEMBERSHIP setsockopt() call is used.)

Note that if network switches which carry the multicast traffic do not support IGMP multicast filtering or do not have it enabled, then the multicast packets may be received by all ports on the switch. Therefore, it is recommended that IGMP multicast filtering be used. Also, if the multicast traffic is required to traverse networks (for example, between the ULAN and the X or Y traffic LANs), then the routers involved must have a multicast routing protocol such as PIM-SM enabled.

2.3.2 Error Handling

Multicast messaging does not require the originator of the messages to be aware of the existence, location or number of recipients. Therefore there is no feedback from the UDS (or other receiving system) to the MPDS. If the UDS receives an invalid message (i.e. XML cannot be parsed, required elements missing, etc.) it will raise an alarm to indicate it has received an invalid message.

2.4 APDT TCP Metadata Interface

This interface is used between the Sirius APDT server and the MPDS. It was originally designed to feed asynchronous PDT updates into the legacy SPACE system to augment synchronous PDT, which was embedded in the AES audio streams. The latter interface was based on AES-18 and was known as SPDT, for Synchronous PDT. Asynchronous PDT comes in two forms: atomic and hybrid. In the atomic case, unreferenced metadata fields are implicitly blank. In the hybrid case, they retain their previous values. Hybrid mode makes it possible to change one or more metadata fields through the APDT server without affecting other SPDT content. For each hybrid update, SPACE provided an atomic update to the APDT server for distribution to other PDT clients (e.g. XM-band uplink and online service providers.)

The SPDT interface to SPACE was eventually deprecated and sources of those feeds were redirected to the APDT server. The APDT server then became the only means of delivering PDT to SPACE.

The UDS replaces the legacy SPACE system, but it does not interface directly to the APDT server. Instead, the MPDS acts as a proxy between the two and therefore assumes the role previously filled by SPACE with regard to this interface. The MPDS uses an XML-S multicast to exchange content with the UDS. That interface format is described in Section 2.3. This section is about the interface between the APDT server and the MPDS.

2.4.1 Connection Specification

In this interface, the MPDS acts as a TCP server and the APDT server acts as the client. Only one APDT server will attempt to connect to the MPDS at any given time, but it will connect to both of the redundant MPDS servers. Both the online and standby MPDS servers will accept connections, but only the online MPDS will forward received content to its client connections.

The APDT server connects to the MPDS on TCP port 1055. On this connection, the APDT server sends APDT messages and the MPDS sends acknowledgements.

The APDT server can also connect to the MPDS on port 1056. On this connection, the MPDS sends out APDT notifications. The APDT server does not send acknowledgements on this connection.

All messages on these connections have an 8-byte message header, with 4 bytes indicating the message type and 4 bytes indicating the message length (in bytes) following the header. Each value is encoded in numeric ASCII form, left padded with zeros (ex: 0015).

2.4.2 Message Format

The template for an APDT message is:

[Type][Length][Payload]

2.4.2.1 Message Type

The *Type* field in the message header is a 4-digit decimal number which defines how to interpret the payload. It is encoded with leading zeroes. The following types are supported by the MPDS:

Message Type	Message Name	Description
0001	Keep-Alive	This message is periodically sent by the APDT server in order to maintain the connection. The payload for this message type is null. The MPDS responds with the same message as an acknowledgement.
0007	APDT Notification	This message is sent by the MPDS to notify the APDT server of the PDT updates from the UDS.
0009	NAK	This message is sent by the MPDS upon receiving an invalid APDT message. The message body contains a textual description of the error.
0015	APDT Update	This message is sent by the APDT server in order to provide current PDT for a channel. The message body contains “apdt” followed by the message details, which are described in the following section. MPDS responds with the same message type and an empty body if the message was valid, or a NAK message if the message was invalid.
0017	Association	Deprecated type. On receipt of a message of this type, the MPDS returns a response with a null payload, but otherwise takes no action.

2.4.2.2 Message Length

The *Length* field in the message header is a 4-digit decimal number corresponding to the number of characters in the message, exclusive of header. It is also encoded with leading zeros.

2.4.2.3 Message Payload

The payload for Keep-Alive messages is always null. The payload for NAK messages is an arbitrary text string describing why the previously received message was rejected by the MPDS. The message payload for an Association message generated by the MPDS is always null. The message payload for an Association message received by the MPDS is arbitrary since the MPDS parses over it. For all others message types, the payload is a series of tags and/or tag-value pairs separated by white space.

White space between a tag and its value is optional. Double quotes are used to preserve white space within a value. Otherwise, all characters are interpreted as literals with the following exceptions:

- Null (zero) characters are illegal.
- Sequences of consecutive white space characters within a quoted value will be collapsed into a single space character.
- Backslash (\) is processed as a literal character except when it precedes a double quote (") or another backslash (\), in which case the (first) backslash is dropped and the second character is treated as a literal. E.g. '\\'->'\', '\"'->'"', but '\a'->'\a'.

The first tag in an APDT Update message is always the literal text “apdt”. It is followed by a channel number in decimal format with a valid range of 1 to 223. The first tag in an APDT Notification message is “notify apdt”. It is also followed by a channel number. In either case, the rest of the payload comprises zero or one instance of each of the following:

Tag	Value (if any)	Description
-a	Artist	Text for the *artist* PDT field for the specified channel.
-t	Title	Text for the *title* PDT field for the specified channel.
-s	PID	Text for the (Sirius) *PID* PDT field for the specified channel.
-d	Artist ID	Text for the *Artist ID* field for the specified channel.
-c	Composer	Text for *Info PDT* field (formerly called *composer*) for the specified channel.
-L		Indicates that the recipient should propagate the message to look-around PDT as well as in-band PDT. The MPDS ignores this option as it is assumed that all updates propagate to LAPDT. In APDT Notification messages sent by MPDS, this option is always specified.
-l	Message ID	Sequence number for cross referencing with other content sources. MPDS ignores this option as it is only required for some SPDT substitution features which are now deprecated. Note that the value in a “notify apdt” message is always arbitrary and has nothing to do with the value in the corresponding “apdt” message.
-U		Indicates that the recipient should use the values provided in this message until the next SPDT update is received. MPDS ignores this option as SPDT is now deprecated.
-z		Specifies hybrid mode, in which unspecified PDT fields are to be left unchanged. If this option is omitted, all unspecified PDT fields are implicitly blank. Hybrid mode is not available for APDT Notification messages.
-A		Indicates that the recipient should start using the most recent SPDT. MPDS will reject the message with NAK if this option is used as SPDT is now deprecated.

Tag	Value (if any)	Description
-P		Indicates that the recipient should switch from APDT to SPDT for this channel. MPDS will reject the message with NAK if this option is used as SPDT is now deprecated.
-g (msg age in msec) -G (msg age in msec)		Indicates the age of the message in milliseconds. This was used to prevent race conditions between SPDT and APDT updates for “blink mode”. MPDS ignores this option as SPDT is now deprecated.

2.4.3 Message Examples

The following are some examples of APDT messages:

00150080apdt 66 -L -I453 -U -a "Thom Rotella" -t "What's the Story?" -s "$O2iC8" -d "sA"

This is an APDT Message for channel 66, with artist “Thom Rotella”, song title “What’s the Story?”, Sirius PID $O2iC8, and artist ID sA.

00150044apdt 112 -z -L -I479 -U -t "HPT 26.01 -1.18"

This is an APDT Message for channel 112. Hybrid mode is specified, so this indicates that the song title for this channel should be changed to “HPT 26.01 -1.18” and all other PDT fields should remain unchanged.

00070070notify apdt 112 -I479 -L -a"Squawk On The Street" -t"HPT 26.01 -1.18"

This is an APDT Notification message for channel 112, with artist “Squawk On The Street”, and song title “HPT 26.01 -1.18”. Note that this is a typical notification that would be produced following the last message. The APDT Notification message always includes the values for all active fields.

This page intentionally left blank.

3.0 MPDS Inbound-Outbound Interface Mapping

3.1 Functional Overview

The MPDS receives metadata content from external sources such as XM Dalet systems in B-4/A4/A5 and DM/DN formats. Other sources use XML-S format. The MPDS can redistribute received content in either format.

3.2 Introduction

The MPDS supports all combinations of input and output formats. If a received message contains fields which are not supported on the output interface, those fields are simply dropped from the outgoing transmission. If a received message does not contain all of the fields that the output interface supports, the missing fields are left blank or set to default values as described in the sections which follow.

The MPDS caches received metadata in a common internal format, which is the union of all supported input and output formats. The mappings between this internal MPDS format and each of the input and output formats are described below.

3.3 Legacy XM Metadata Input Mapping

The MPDS can be configured to use either the DM/DN messages or the A4/A5 messages for legacy XM metadata input. If the MPDS is configured to use the DM/DN message, each time a DM/DN message is received by the MPDS, a metadata message is generated (with the appropriate queue order value) as the DM message contains both the artist and song information. If the MPDS is configured to use the artist/song messages instead, then if a B-4 message is received the MPDS waits a configurable interval (nominally 5 seconds) for an A4/A5 pair and if none is received uses the B-4 data to construct the metadata contents. If an A4/A5 message is received within 5 seconds, then the metadata message is constructed using the contents of the A4/A5 message. Note that if the artist/song messages are used then only queue order 0 (current song) data can be forwarded. The channel code value for the metadata message is extracted from the configuration of the input channel in the MPDS as specified by the MPDS operator.

The MPDS forwards one output message for each input message. This means that if repeated messages are received from its input source (such as the Dalet system) then each copy will be sent to the destination server(s).

The following tables show the source and destination fields for DM messages or artist/song messages being forwarded to XML-S outputs:

Table 3-1 A4/A5 to MPDS Mapping

MPDS Field	A4/A5 Field	Constraints and Conversions
artist	Artist Label	Truncated to 36 characters.
song_title	Song Label	Truncated to 36 characters.

Table 3-2 B-4 to MPDS Mapping

MPDS Field	B-4 Field	Constraints and Conversions
artist	Artist Label	None (16 characters maximum in message)
program_id	Program ID	None
event_id	n/a	Current time and configured machine ID for the channel. The event_id will be generated by using the program ID and prepended date string.
song_title	Song Label	None (16 characters maximum in message)

Table 3-3 DM/DN to MPDS Mapping

MPDS Field	DM/DN Field	Constraints and Conversions
album	album	Truncated to 36 characters maximum.
artist	artist	Truncated to 36 characters maximum.
artist_id	artist_id	Base-64 to decimal conversion. Defaults to null if received content is invalid or missing.
composer	composer	Truncated to 36 characters maximum.
duration	duration	None
music_db_id	music_db_id	None
program_id	program_id	None
sirius_pid	sirius_pid	Base-64 to decimal conversion. Defaults to null if received content is invalid or missing.
song_title	song_title	Truncated to 36 characters maximum.
ts	start_offset	Use now + start_offset to set the ts field. Note that only one of ts or start_offset is required, ts is preferred. URL reserved characters escaped. May be truncated if message length exceeds 2000 characters.
start_offset	start_offset	URL reserved characters escaped. May be truncated if message length exceeds 2000 characters.

3.4 XML-S Metadata Input Mapping

Table 3-4 XML-S to MPDS Mapping

MPDS Field	XML-S Field	Constraints and Conversions
album	${Album}	XML reserved characters un-escaped.
artist	${Artist}	XML reserved characters un-escaped
artist_id	${Artist ID}	XML reserved characters un-escaped. Base-64 to decimal conversion. Defaults to null if received content is invalid or missing.
composer	${Composer}	XML reserved characters un-escaped.
duration	${Duration}	XML reserved characters un-escaped.
event_id	N/A	Current time and configured machine ID for the channel. The event_id will be generated by using the program ID and prepended date string.
layer	${Layer}	XML reserved characters un-escaped.
music_db_id	${Music DB ID}	XML reserved characters un-escaped.
program_id	${Program ID}	XML reserved characters un-escaped.

Table 3-4 XML-S to MPDS Mapping

MPDS Field	XML-S Field	Constraints and Conversions
Record_enabled	${Record Enabled}	XML reserved characters un-escaped.
sirius_pid	${Sirius PID}	XML reserved characters un-escaped. Base-64 to decimal conversion. Defaults to null if received content is invalid or missing.
song_title	${Song Title}	XML reserved characters un-escaped.
start_offset	${Start Offset}	XML reserved characters un-escaped.
timestamp	${UTC Time Stamp}	XML reserved characters un-escaped.

3.5 APDT Input Mapping

Note that for APDT inputs, received hybrid mode messages will result in the MPDS combining the previously received metadata for the input with the provided fields in order to generate a complete metadata update, which will then be forwarded to the configured outputs.

Fields with options not provided in the input APDT message will be treated as blank.

Table 3-5 APDT to MPDS Mapping

MPDS Field	APDT Option	Constraints and Conversions
artist	-a	None
artist_id	-d	Base-64 to decimal conversion. Defaults to null if received content is invalid or missing.
composer	-c	None
event_id	N/A	Current time and configured machine ID for the channel. The event_id will be generated by using the PID and prepended date string.
program_id	N/A	Automatically generated by MPDS.
sirius_pid	-s	Base-64 to decimal conversion. Defaults to null if received content is invalid or missing.
song_title	-t	None

3.6 Legacy XM Metadata Output Mapping

For UDS metadata outputs, when processing messages with a queue order of 0 (current song), the B-4 message is constructed from the first 16 characters of the artist and song labels and the A5 and A4 messages are constructed from the first 36 characters of the corresponding labels. DM and DN messages are also generated with the available message data. Heartbeat (H0) messages are generated periodically, every 10 seconds.

For queue order 0, the MPDS sends an A4, A5, B-4, and DM message for each forwarded metadata message, always sending the B-4 message first, following it immediately with a repeat of the B-4 message, then a single A4 message, followed by a single A-5 message and lastly the DM message followed by a repeat of the DM message. (Message repeats are used to mitigate the effects of possible message corruption downstream of the MPDS, such as due to Klotz switching for commercial insertion, etc.) When processing messages with a queue order of 1 (next song), two copies of the corresponding DN message will be generated. Messages with a queue order of greater than 1 are currently not forwarded on this interface.

The following tables list the mappings to the A4, A5, B-4, and DM metadata messages:

Table 3-6 MPDS to A4 Mapping

A4 Field	MPDS Field	Constraints and Conversions
Song Title	song_title	None

Table 3-7 MPDS to A5 Mapping

A5 Field	MPDS Field	Constraints and Conversions
Artist	artist	None

Table 3-8 MPDS to B-4 Mapping

B-4 Field	MPDS Field	Constraints and Conversions
Advanced Application Control	record_enabled	None
Duration/Progress Format	N/A	Always set to 0.
Duration and Progress	N/A	Always set to 000000.
Display 1 Size	N/A	Always set to 8.
Display 2 Size	N/A	Always set to 10.
Artist Mask 1	N/A	Always set to 11111111.
Artist Mask 2	N/A	Always set to 1111111111.
Artist Label	artist	First 16 characters.
Song Mask 1	N/A	Always set to 11111111.
Song Mask 2	N/A	Always set to 1111111111.
Song Label	song_title	First 16 characters.
Program ID	program_id	None

Table 3-9 MPDS to DM/DN Mapping

DM/DN Field	MPDS Field	Constraints and Conversions
album	album	URL reserved characters escaped. May be truncated if message length exceeds 2000 characters.
artist	artist	URL reserved characters escaped. May be truncated if message length exceeds 2000 characters.
artist_id	artist_id	URL reserved characters escaped. Decimal to base-64 conversion. Defaults to null if received content is invalid or missing.
composer	composer	XML reserved characters un-escaped. URL reserved characters escaped. May be truncated if message length exceeds 2000 characters. If element not present in XML-S message, DM contains only the tag and no value e.g. “composer=”.
description	description	URL reserved characters escaped. May be truncated if message length exceeds 2000 characters. If element not present in XML-S message, DM contains only the tag and no value e.g. “description=”.
event_id	event_id	URL reserved characters escaped. May be truncated if message length exceeds 2000 characters.
isrc	isrc	URL reserved characters escaped. May be truncated if message length exceeds 2000 characters. If element not present in XML-S message, DM contains only the tag and no value e.g. “isrc=”.
item_code	item_code	URL reserved characters escaped. May be truncated if message length exceeds 2000 characters. If element not present in XML-S message, DM contains only the tag and no value e.g. “item_code=”.
label	label	URL reserved characters escaped. May be truncated if message length exceeds 2000 characters. If element not present in XML-S message, DM contains only the tag and no value e.g. “label=”.
music_db_id	music_db_id	URL reserved characters escaped. May be truncated if message length exceeds 2000 characters.
program_id	program_id	URL reserved characters escaped. Decimal to base-64 conversion. Defaults to null if received content is invalid or missing.
sirius_pid	sirius_pid	URL reserved characters escaped. May be truncated if message length exceeds 2000 characters. If element not present in XML-S message, DM contains only the tag and no value e.g. “sirius_pid=”.
sku	sku	URL reserved characters escaped. May be truncated if message length exceeds 2000 characters
song_title	song_title	URL characters escaped. May be truncated if message length exceeds 2000 characters.
title_id	title_id	URL reserved characters escaped. May be truncated if message length exceeds 2000 characters
duration	duration	URL reserved characters escaped. May be truncated if message length exceeds 2000 characters.
start_offset	timestamp	Use ts - now to set the start_offset field. URL reserved characters escaped. May be truncated if message length exceeds 2000 characters.
start_offset	start_offset	URL reserved characters escaped. May be truncated if message length exceeds 2000 characters.

3.7 XML-S Metadata Output Mapping

Table 3-10 MPDS to XML-S Mapping

XML-S Field	MPDS Field	Constraints and Conversions
${Album}	album	Reserved XML characters escaped.
${Artist}	artist	Reserved XML characters escaped.
${Artist ID}	artist_id	Reserved XML characters escaped. Decimal to base-64 conversion. Defaults to null if received content is invalid or missing.
${Bus Code}	N/A	Always set to value configured in MPDS. Maximum 50 characters (normally 6). Reserved XML characters escaped, no truncation
${Composer}	composer	Reserved XML characters escaped, truncated to 36 characters maximum.
${Duration}	duration	Reserved XML characters escaped.
${Layer}	layer	Reserved XML characters escaped.
${Music DB ID}	music_db_id	Reserved XML characters escaped.
${Program ID}	program_id	Reserved XML characters escaped.
${Queue Order}	N/A	Set to the queue order of the message.
${Sirius PID}	sirius_pid	Reserved XML characters escaped. Decimal to base-64 conversion. Defaults to null if received content is invalid or missing.
${Song Title}	song_title	Reserved XML characters escaped.
${SID}	N/A	Always set to value configured in MPDS. Reserved XML characters escaped.
${Start Offset}	start_offset	Reserved XML characters escaped.
${UTC Time Stamp}	timestamp	Reserved XML characters escaped.

3.8 APDT Metadata Output Mapping

Note that for APDT messages output from the MPDS, all applicable metadata fields are always included in the message (i.e. hybrid mode is not used). If any field is empty, the corresponding option is left out of the message. Only messages with queue order 0 are forwarded on this interface.

Table 3-11 MPDS to APDT Mapping

APDT Option	MPDS Field	Constraints
-a	artist	None
-d	artist_id	Decimal to base-64 conversion. Defaults to null if received content is invalid or missing.
-c	composer	None
-s	sirius_pid	Decimal to base-64 conversion. Defaults to null if received content is invalid or missing.
-t	song_title	None

4.0 XM Program ID (XMPID) Generation and Handling

4.1 General Operations

This section contains a general outline of the usage of the XMPID on the High Band system. For details of the XMPID *contents*, refer to AD1 PID (Program ID) Coding for XM Band.

Some high-band receivers support record and playback. The XM Program ID (aka. XMPID) field identifies boundaries between programs (or songs) for playlist generation and playback control. The receiver starts a new file (or "track") when the value changes. The actual value is not important except that duplicate values within a recorded session can be problematic for playback control.

There are certain situations in which it is desirable to avoid triggering a new file/track creation when other metadata (such as artist and song labels) changes. Examples are:

- Interstitial content (jingles, etc) within broadcasts for which it is desirable to include this material as part of a single track on the receiver.
- Rotating metadata (upcoming games, impromptu metadata changes generated by DJ's during content broadcasting).
- Sports and news tickers.

Additionally, it is important that players do not include interstitial material (DJ banter, station identification messages, jingles) as part of a track saved when the user is recording typical commercial music material. For example, if a user is recording a song, any station identification message that may either precede or follow the content should not be recorded as part of the track identified as the song title when saved to the receiver.

A recording will contain interstitial material if it is sent as part of the audio track during playback but will not create a change of XMPID. An example would be a DJ starting the play out of a song and talking over the beginning of the song.

To avoid unnecessary recording breaks, an XMPID of 0 is sent in the artist/song label message (B4) from broadcast for the cases where it is undesirable to place a break in the recording. This requires broadcast to send a PID of 0 in B4 messages for content that should be included in a single track recording (such as rotating metadata, sports scores, and certain interstitial content).

In the event the UDS receives an XMPID of 0, the artist/song label messages in the BIC will continue using the previous XMPID value until it receives a B4 message from broadcast that contains a non-zero value. This prevents the receiver from closing and starting a new file. An example of this situation is a baseball game that is updated with a new artist/song label message (to show the current score) but to continue the same XMPID value to ensure the entire game is recorded as a single track.

Future implementations of this strategy may be used to place intelligent transitions during certain broadcasts, such as baseball games (divide recordings into inning segments) and live events (divided by band playing).

4.2 Requirements and Rules

The subsystems that deal with XMPID generation and handling must adhere to the following rules and concepts.

4.2.1 XMPID generation - Programming Center

1. The XMPID is generated automatically by the Dalet, Harris, or APDT Server (they are not generated by Prophet) systems and sent as metadata with each element as it is played.

2. Every element will have an XMPID.

3. The XMPID will be delivered to the UDS via the B-4 (BIC Artist/Song Label) message as well as the A1 (Program ID) message.

4. Advertisements will generally maintain a valid XMPID all the way to the radio so that they can be indexed on radios.

5. Rotating metadata messages (e.g. information about upcoming games) and sports scores will be delivered with an XMPID of 0.

6. If the broadcast systems allow, elements such as interstitial material, jingles, etc will be delivered to the uplink with an XMPID of 0, except for the interstitial material between songs on the music channels typically used by player users to record commercial music tracks, which will be delivered with a valid XMPID.

7. Pre-recorded, voice tracks do not send an XMPID.

4.2.2 XMPID Handling – Uplink

1. The UDS will receive a PID via the B-4 message or the DM/DN message depending on its configuration.

2. The UDS will replace an XMPID of 0 with the XMPID from the previous message currently being repeated in the BIC.

3. The MPDS will forward all DM/DN messages even if they contain an XMPID of 0.

4.2.3 PID Handling - Receiver

1. A radio will generate a new file transition based on a change of XMPID regardless if there is a change to artist/label message.

2. A radio will not generate a new file transition if a new artist/song label arrives with the same XMPID as the previous message but will display the new labels if the sequence number is incremented.